# Robotics Programming Challenge 2

## 1. Introduction

You will implement the velocity and acceleration kinematics for the Franka Emika Panda, a 9-DoF arm from Challenge 1. Additionally, you will be implementing a trajectory planning method for a specific task.

## 2. The setup

You will use the same `panda_kinematics` package you created in Challenge 1.

- A new file called `trajectory_node.py` will be provided to you. Place it inside the inner `panda_kinematics` folder.
- You must add a new entry point in your `setup.py` so ROS 2 knows how to run it:
  - `'trajectory_node = panda_kinematics.trajectory_node:main'`

## 3. The Code Template

The `trajectory_node.py` file contains the ROS 2 infrastructure (subscribers, publishers, and the execution loop).

- Three new functions need to be implemented in the declared functions shown below:
  - `def compute_jacobian(q):`
  - `def compute_jacobian_dot(q, dq):`
  - `def plan_trajectory(q_start, q_end, T, dt):`
- Do not modify the infrastructure below the marked line.

# 4. Requirements

- Fill in `compute_jacobian(q)`. Because you are only dealing with linear velocity (x, y, z), this will be a 3x7 matrix, not a 6x7 matrix.
- Fill in `compute_jacobian_dot(q, dq)`. You can compute the derivative of the jacobian analytically or numerically (finite differences). Both are acceptable.
- Fill in `plan_trajectory(q_start, q_end, T, dt)`. It must return four arrays of equal length: times, joint positions, joint velocities, and joint accelerations.
- How you generate these is up to you. You can use joint-space planning (e.g., quintic or cubic polynomials), task-space planning , or another method if you can defend it.
- In Challenge 1, you changed `mujoco_ros2` to `POSITION` mode. For this challenge, you must test your trajectories under both `POSITION` and `VELOCITY` control modes.
- You will need to recompile the bridge or edit the launch file to switch between modes.
- For `POSITION` mode, publish the planned joint positions (`q_array`).
- For `VELOCITY` mode, publish the planned joint velocities (`dq_array`). You will need to write a small integrator in your node to accumulate the velocities, or rely on the bridge's internal integration.
- Observe how the robot tracks the trajectory in both modes

# 5. Changing the control mode:

To switch modes, you must modify the external ROS 2 files, not your Python code.

1. Open the `mujoco_ros2` launch file (`example.py`) and change the parameter from `{"control_mode": "POSITION"}` to `{"control_mode": "VELOCITY"}`.
2. Rebuild the bridge: `colcon build --packages-select mujoco_ros2`

Switching the Python Template (Your Code):

The bridge expects whatever you publish on `/joint_commands` to match its current mode. In your `trajectory_node.py`, look at the `tick()` method (in the `INFRASTRUCTURE` section). You will find commented lines that let you swap what is being published without changing any other logic:

- For POSITION mode: Ensure the node is publishing `self.q_traj[self.step_idx]`.
- For VELOCITY mode: Uncomment `q_target = self.dq_traj[self.step_idx]` and comment out the position line.

What to Observe:

Run your trajectory in both modes. Watch the MuJoCo viewer and note the differences:

- Does the robot track the trajectory more closely in one mode than the other?
- In `VELOCITY` mode, since you are relying on the simulator to integrate the velocities you send, does the arm drift or lag more toward the end of the trajectory?
- Record your observations, as you will need them for the evaluation reflection.

# 6. Team's Variant

Every team is required to implement the exact same mathematical requirements (Linear Jacobian, Jacobian Derivative, and Trajectory Planning) and test them in both control modes. To make the debate meaningful, each team is assigned a specific descriptive scenario.

You will define the specific numbers for your start and end joint configurations in your code based on your team's scenario. You are responsible for ensuring the chosen configurations are reachable and within joint limits.

- Team 1 — The Standard Pick-and-Place
  Your robot starts in a neutral, central posture and moves to a forward-reaching configuration, as if picking an object from a table directly in front of it and moving it slightly upwards and inward. All joints move by moderate amounts.
  *Debate Focus:* How did you verify that your planned trajectory does not overshoot the joint limits during the motion, given the moderate but simultaneous movement of all joints?

- Team 2 — The Cartesian Path
  Your robot must move its end-effector in a straight line through Cartesian space from its starting point to a nearby ending point.
  *Debate Focus:* Your joint velocities at the boundaries behave differently from the teams that interpolated joint angles directly. Why? What are the kinematic trade-offs of enforcing a straight line in Cartesian space compared to joint space?

- Team 3 — The Wide Arc
  Your robot starts with its arm positioned to the left of its base and must move to a configuration on the right side. The trajectory requires a broad, sweeping motion across the front of the robot.

*Debate Focus:* During this large sweep, the required joint velocities vary significantly across the joints. How does your trajectory planner ensure that no single joint approaches its velocity or acceleration limits during the sweep?

- Team 4 — The Vertical Transfer
  Your robot starts with the end-effector positioned low and must move it to a high position, keeping the orientation of the gripper relatively constant relative to the ground.
  *Debate Focus:* When moving vertically while maintaining orientation, several joints must compensate for each other. How does your choice of planning space (joint vs. task) affect the coordination of these joints?

- Team 5 — The Folded Approach
  Your robot starts in an extended configuration and must move to a folded configuration where the end-effector is brought close to the robot's base.
  *Debate Focus:* As the arm folds in on itself, the distance between the links decreases. How did you check that the planned joint angles do not result in self-collision, and what would you do if they did?

- Team 6 — The High Reach
  Your robot starts in a low posture and must reach upwards and outwards to a position near the top of its workspace.
  *Debate Focus:* As the arm approaches the upper boundary of its workspace, it gets closer to kinematic singularities. How did you detect this in your trajectory, and how did you adjust the motion to prevent the required joint velocities from becoming excessive?

# 7. Evaluation:

The evaluation for this challenge runs alongside the milestone 3 evaluation on **Wednesday, 11th of November**.

Every team member must be present as the evaluation is individual. You will each be asked questions about your own implementation and your own reasoning, and you will be graded on your own answers. A teammate who cannot explain a design decision made in the shared code does not earn credit for it.

What you expect to be asked:

- To derive a column of the geometric Jacobian from your DH table.
- To explain what the computed matrices physically represent.
- To justify your trajectory method and to walk through its behaviour at the boundary conditions.
- To describe what happens to your trajectory at or near a singularity, and to defend how you handled it.
- To respond to at least one counterargument about a design decision you made.

**Timings will be announced separately.**